Redis 分布式锁原理简述

概述

分布式锁是一种在分布式系统中用于控制多个进程或服务对共享资源访问的机制。Redis 作为一种高性能的键值存储系统，因其高并发处理能力和简单易用的特性，被广泛应用于实现分布式锁。本文将详细探讨基于 Redis 实现分布式锁的基本原理、常见问题及解决方案。

基本原理

锁的获取与释放

Redis 实现分布式锁的核心命令是 SETNX 和 DEL。SETNX 命令用于设置一个键值对，只有当键不存在时才会成功设置，并返回 1；如果键已存在，则返回 0。通过这种方式，可以确保在同一时刻只有一个客户端能够成功获取锁 [3]。

```
Long result = jedis.setnx("lockKey", "lockValue");
if (result == 1) {
    // 获取锁成功
}
```

为了防止死锁，通常会为锁设置一个过期时间。早期的做法是使用 EXPIRE 命令来设置过期时间，但这种方法存在原子性问题。因此，Redis 2.6.12 版本引入了 SET 命令的 NX 和 EX 选项，可以在一条命令中同时设置键值和过期时间 [13]。

```
jedis.set("lockKey", "lockValue", SetParams.setParams().nx().ex(3));
```

锁的释放

释放锁时需要确保删除的是当前客户端持有的锁，避免误删其他客户端的锁。为此，可以使用 Lua 脚本来保证判断和删除操作的原子性 [1]。

```
if redis.call('get', KEYS[1]) == ARGV[1] then
    return redis.call('del', KEYS[1])
else
```

```
    return 0
end
```

常见问题及解决方案

锁超时问题

如果业务执行时间超过锁的过期时间，锁会被自动释放，导致其他客户端可以重新获取锁，从而破坏互斥性。为了解决这个问题，可以采用锁续命（Lock Renewal）机制。Redisson 提供了看门狗（Watchdog）机制，自动延长锁的有效时间 [2]。

看门狗机制

看门狗机制的核心思想是：只要业务代码还在执行，就每隔一段时间自动延长锁的过期时间。默认情况下，Redisson 的 `lockWatchdogTimeout` 设置为 30 秒，每 10 秒检查一次并续期 [2]。

```
private void scheduleExpirationRenewal(long threadId) {
    ExpirationEntry entry = new ExpirationEntry();
    ExpirationEntry oldEntry = EXPIRATION_RENEWAL_MAP.putIfAbsent(getEntryN
    if (oldEntry != null) {
        oldEntry.addThreadId(threadId);
    } else {
        entry.addThreadId(threadId);
        renewExpiration();
    }
}

private void renewExpiration() {
    ExpirationEntry ee = EXPIRATION_RENEWAL_MAP.get(getEntryName());
    if (ee == null) {
        return;
    }
    Timeout task = commandExecutor.getConnectionManager().newTimeout(timeou
        RFuture<Boolean> future = renewExpirationAsync();
        future.onComplete((res, e) -> {
            if (res && ee.getThreadId() != null) {
                renewExpiration();
```

```
            }
        });
    }, internalLockLeaseTime / 3, TimeUnit.MILLISECONDS);
    ee.setTimeout(task);
}
```

RedLock 算法

RedLock 是一种在多实例 Redis 中实现分布式锁的算法，旨在提高锁的可靠性。RedLock 通过在多个独立的 Redis 实例上依次尝试加锁，只有当大多数实例（N/2+1）成功加锁且总耗时小于锁的有效时间时，才认为加锁成功 [7]。

```
Config config01 = new Config();
config01.useSingleServer().setAddress("redis://192.168.0.100:6379");
RedissonClient redissonClient01 = Redisson.create(config01);

Config config02 = new Config();
config02.useSingleServer().setAddress("redis://192.168.0.100:6380");
RedissonClient redissonClient02 = Redisson.create(config02);

Config config03 = new Config();
config03.useSingleServer().setAddress("redis://192.168.0.100:6381");
RedissonClient redissonClient03 = Redisson.create(config03);

RLock lock01 = redissonClient01.getLock("myLock");
RLock lock02 = redissonClient02.getLock("myLock");
RLock lock03 = redissonClient03.getLock("myLock");

RedissonRedLock redLock = new RedissonRedLock(lock01, lock02, lock03);
boolean isLock = redLock.tryLock(10, 300, TimeUnit.SECONDS);
if (isLock) {
    // 处理业务逻辑
}
```

总结

基于 Redis 实现分布式锁是一种高效且可靠的方法。通过 SETNX 和 DEL 命令结合 Lua 脚本，可以实现基本的锁机制。为了应对锁超时问题，Redisson 提供了看门狗机制，自动延

长锁的有效时间。此外，RedLock 算法通过在多个独立的 Redis 实例上加锁，进一步提高了分布式锁的可靠性。这些方法和技术使得 Redis 成为实现分布式锁的理想选择 [1][2][3][7][13]。

参考文献

[1] juejin.cn — Redis高级（八）、全网最详细的分布式锁原理+实战+WatchDog机制源码解读首先搞明白RedLock与Rediss - 掘金 URL: https://juejin.cn/post/7192071135646646332 原文片段：「Redis高级（八）、全网最详细的分布式锁原理+实战+WatchDog机制源码解读首先搞明白RedLock与Rediss - 掘金 Redis高级（八）、全网最详细的分布式锁原理+实战+WatchDog机制源码解读 JavaLyHn 2023-01-24 1,442 阅读10分钟 觉得对你有益的小伙伴记得点个赞+关注 后续完整内容持续更新中 希望一起交流的欢迎发邮件至 javalyhn@163.com 1. Distributed locks with Redis官网说明 2. 使用场景 多个服务间保证同一时刻同一时间段内同一用户只能有一个请求(防止关键...」

[2] blog.csdn.net — Redis分布式锁经典问题详解：误删、过期、续命，及Redisson看门狗机制_redisson 设置了过期时间会自动续期吗-CSDN博客 URL: https://blog.csdn.net/qq_21982453/article/details/153468955 原文片段：「Redis分布式锁经典问题详解：误删、过期、续命，及Redisson看门狗机制_redisson 设置了过期时间会自动续期吗-CSDN博客 Redis分布式锁经典问题详解：误删、过期、续命，及Redisson看门狗机制 原创 已于 2026-03-19 11:37:07 修改 · 1.2k 阅读 · 本内容遵循CC 4.0 BY-SA版权协议 版权声明：本文为博主原创文章，遵循 CC 4.0 BY-SA 版权协议，转载请附上原文出处链接和本声明。 GEO检测 · 收录于 Java 当前文章被收录于： Java 41 篇文章 0 人学习 Java相关内容...」

[3] cnblogs.com — 聊聊如何用 Redis 实现分布式锁？ - 小林coding - 博客园 URL: https://www.cnblogs.com/xiaolincoding/p/16517673.html 原文片段：「聊聊如何用 Redis 实现分布式锁？ - 小林coding - 博客园 小林coding 公众号：「小林coding」酷爱图解晦涩难懂的计算机基础知识。 聊聊如何用 Redis 实现分布式锁？ 作者：小林coding 计算机八股文网站： https://xiaolincoding.com 哈喽，我是小林。 今天跟大家聊聊两个问题： 如何用 Redis 实现分布式锁？ Redis 是如何解决集群情况下分布式锁的可靠性问题的？ 如何用 Redis 实现分布式锁的？ 分布式锁是用于分布式环境下并发控制的一种机制，用于控制某个资源在同一时刻只能被一个应用所使用...」

[4] javaguide.cn — 分布式锁实现方案详解：Redis、Redlock、ZooKeeper 与 Redisson 看门狗 | JavaGuide URL: https://javaguide.cn/distributed-system/

distributed-lock-implementations.html 原文片段：「分布式锁实现方案详解：Redis、Redlock、ZooKeeper 与 Redisson 看门狗 | JavaGuide 跳至主要内容 项目介绍 面试准备（必看） Java 数据库 开发工具 常用框架 系统设计 分布式 分布式系统知识体系 分布式系统入门 分布式高频面试题 理论&算法&协议 API网关 API网关基础知识总结 Spring Cloud Gateway面试题总结 分布式ID 分布式ID生成方案详解 分布式ID设计实战指南 分布式锁 分布式锁入门介绍 分布式锁常见实现方案总结 分布式事务 分布式事务解决方案总结 分布式配置中心 分布式配置...」

[5] developer.jdcloud.com — 深入理解分布式锁：原理、应用与挑战-京东云开发者社区 URL: https://developer.jdcloud.com/article/3777 原文片段：「深入理解分布式锁：原理、应用与挑战-京东云开发者社区 您好！ 欢迎来到京东云开发者社区 登录 首页 博文 课程 大赛 工具 用户中心 开源 首页 博文 课程 大赛 工具 开源 更多 用户中心 开发者社区 > 博文 > 深入理解分布式锁：原理、应用与挑战 分享 打开微信扫码分享 点击前往QQ分享 点击前往微博分享 点击复制链接 深入理解分布式锁：原理、应用与挑战 liuhaozzu 2024-04-30 IP归属：北京 4799浏览 前言 在单机环境中，我们主要通过线程间的加锁机制来确保同一时间只有一个线程能够访问某个共享资源或执行某个关键代码块，...」

[6] oneuptime.com — How to Implement Distributed Locks with Redis (Redlock) URL: https://oneuptime.com/blog/post/2026-01-21-redis-distributed-locks/view 原文片段：「How to Implement Distributed Locks with Redis (Redlock) Skip to main content How to Implement Distributed Locks with Redis (Redlock) A comprehensive guide to implementing distributed locks with Redis using the Redlock algorithm, including lock safety, automatic expiration. By @na...」

[7] thediligentengineer.com — The Redlock Algorithm | TDE URL: https://thediligentengineer.com/the-redlock-algorithm 原文片段：「The Redlock Algorithm | TDE The Redlock Algorithm The Diligent Engineer · Apr 21, 2025 · 4 min read The Redlock algorithm is a distributed locking mechanism designed for Redis to ensure mutual exclusion across clients in a distributed system. It operates by acquiring locks on a m...」

[8] redis.io — Distributed Locks with Redis | Docs URL: https://redis.io/docs/latest/develop/clients/patterns/distributed-locks 原文片段：「Distributed Locks with Redis | Docs {"categories":["docs","develop","stack","oss","rs","rc","oss","kubernetes","clients"],"description":"A distributed lock pattern with Redis\n","duplicateOf":"head:data-ai-metadata","location":"body","title":"Distributed Locks with Redis","tableO...」

[9] redis.io — Redis lock: fast distributed locking explained URL: https://redis.io/glossary/redis-lock 原文片段：「Redis lock: fast distributed locking explained 97% believe in it. 4% have built it. New research: State of context engineering. Read the report Resource Center Events & webinars Blog Videos Glossary Resources Architecture Diagrams Demo Center Resource Center Events & webinars Blo...」

[10] youtube.com — How Distributed Lock works | ft Redis | System Design - YouTube URL: https://www.youtube.com/watch?v=qY4MfWv01pI 原文片段：「How Distributed Lock works | ft Redis | System Design - YouTube About Press Copyright Contact us Creators Advertise Developers Terms Privacy Policy & Safety How YouTube works Test new features NFL Sunday Ticket © 2026 Google LLC」

[11] github.com — Full-Stack-Notes/notes/Redis_分布式锁原理.md at master · heibaiying/Full-Stack-Notes · GitHub URL: https://github.com/heibaiying/Full-Stack-Notes/blob/master/notes/Redis_%E5%88%86%E5%B8%83%E5%BC%8F%E9%94%81%E5%8E%9F%E7%90%86.m 原文片段：「Full-Stack-Notes/notes/Redis_分布式锁原理.md at master · heibaiying/Full-Stack-Notes · GitHub Skip to content You signed in with another tab or window. Reload to refresh your session. You signed out in another tab or window. Reload to refresh your session. You switched accounts on anot...」

[12] seven97.top — Redis分布式锁：从SETNX到Redisson的演进 | Seven的菜鸟成长之路 URL: https://seven97.top/database/redis/05-implementdistributedlocks.html 原文片段：「Redis分布式锁：从SETNX到Redisson的演进 | Seven的菜鸟成长之路 跳至主要内容 Java 数据库 MySQL Redis Redis数据类型全攻略：String、List、Hash、Set的实战应用 Redis是如何高效存储与访问数据？ Redis命令处理机制源码探究 Pipelining原理：批量命令背后的秘密 Redis究竟为什么这么快？ Redis持久化：从AOF到RDB，如何实现数据不丢失？ Redis缓存管理：过期删除策略的深度剖析 Redis内存淘汰策略：如何高效管理有限内存？ Redis主从复制是如何保障数据一致性的？...」

[13] support.huaweicloud.com — support.huaweicloud.com URL: https://support.huaweicloud.com/intl/zh-cn/redisug-nosql/redis_bestpractice_2_010.html